Chart Clustering: Implementation, Scoring and Refresh

*Companion to "Chart Clustering: Concept Note" (6 Nov 2026). The concept note says **what** the chart types are and how a chart gets its answer. This note says **how it is built in this repo**, **how every number is computed**, and **how the model is refreshed**. Section numbers in brackets (§) point to the concept note.*

---

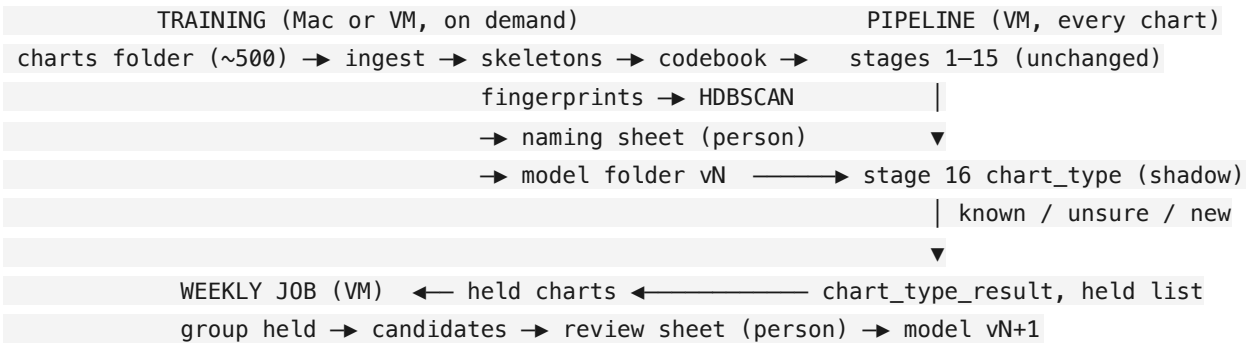
1. In one paragraph

A new, separate training track, `training/chart-clustering/`, sits beside the page-leaf and encounter trainers. You give it **a folder of about 500 charts**, shaped like `core-pipeline/outputs/charts/`. Only each page's Tesseract text is required; images and the pipeline's CSVs are used when present (§2). It builds the note-look codebook and the chart-type catalogue from them, and writes one versioned model folder. The pipeline loads that folder in a new last stage, `chart_type` (stage 16). That stage runs **in parallel with everything else, in shadow**:

- it reads what stages 1–15 already wrote;
- it never changes any of their outputs;
- it only adds its own rows.

New charts are matched against the catalogue. Charts that match nothing are held. A weekly job groups the held charts into candidate types for a person to review. A full rebuild from a new or larger folder is the quarterly refresh.

The fingerprint is built from Tesseract text first, not from images as in the concept note. A note's template (headers, field labels, EMR boilerplate) is in its text, every production chart has Tesseract text for every page, and images are optional in your folder. Images add a block only when nearly all training charts have them (§2, §3.2).



---

2. Training input: a folder of about 500 charts

The trainer takes **one argument, the folder**: `--root <folder>`. The folder has the same format as `core-pipeline/outputs/charts/` (one subfolder per chart, as the pipeline writes it), so you can point it at any copy of that folder, or at `core-pipeline/outputs/charts/` itself. The trainer only reads it and writes nothing into it. **Only the Tesseract text is required.**

```
<training root>/
├── charts.csv                          optional: split, label, client / project, rr_method
└── 52743839_44976074/
    ├── ocr/52743839_44976074_prelim.txt     REQUIRED  Tesseract text ("===== 7.jpg =====")
    │   (or text/<page>.txt, one file per page)
    ├── pages/*.jpg  corrected-pages/*.jpg   optional  page images (corrected copy preferred)
```

```
    ├── imaging/<chart>_final.csv            optional  page type, specialty, blank/junk …
    ├── imaging/<chart>_hw_printed.csv       optional  handwritten / printed per page
    └── imaging/<chart>_quality.csv          optional  input DPI
```

What the folder has	What the trainer does
Text only (the minimum)	Runs the pipeline's own page-leaf model and keyword canon on each page's Tesseract text to get page type and specialty. Blank/junk = the pass-1 rules on the text. Duplicates = the same word-overlap rule as the pipeline. Capture traits come from the text (§3.2)
Text + `imaging/*.csv` (charts run through the pipeline)	Takes page type, specialty, blank/junk, duplicates and handwriting from the CSVs. That is faster and identical to production
Text + images	Adds the image look (DINOv2-small) and the handwriting model, **only if at least 90% of the charts have images**. Otherwise images are ignored, so every chart is fingerprinted the same way

Why text first works: every chart in production has the same Tesseract text (stage 2, every page). A fingerprint built from Tesseract text in training is therefore built exactly the same way in production. The page-leaf model was also trained on Tesseract text. The Final 1 and Final 2 OCR are never used for the fingerprint, even when present, because a chart without them would look different.

Size check for 500 charts:

- about 500 × 60 pages = 30,000 pages, of which about 10,000 are Progress Notes;
- page classification on CPU takes about 25 minutes on the VM (16 cores), or about 50 on the Mac;
- the text embeddings take about 5 minutes;
- with images, add about 25 minutes for DINOv2 on the VM.

500 charts is a good size: expect about 15–40 types at at least 3 charts each, plus a noise pool of 10–30%.

`charts.csv` (optional).

- Without it, every chart is `train`, apart from a fixed-seed 20% kept as `val`.
- `label` is your own family name where you know one. It is only used for scoring (§6.2).
- Without `client_id` / `project_id` / `rr_method`, the source and retrieval parts get weight 0. The other weights are scaled up, so the notes still carry everything.

  Any folder works. `--root core-pipeline/outputs/charts` trains straight on the pipeline's working folders. One thing to know: chart folders there that have not been run for 7 days are deleted by the outputs purge (only inside `core-pipeline/outputs/`). To keep a training set for re-runs, copy it anywhere else (for example `/Volumes/Extreme SSD/Advantmed/Training/charts/`) and pass that path instead. Nothing in the trainer changes.

---

3. Implementation

3.1 Where the code goes

```
training/chart-clustering/        (GitHub only, never on the VM, like the other trainers)
├── README.md
├── requirements.txt          scikit-learn, hdbscan, sentence-transformers, openpyxl (+ torch CPU)
├── ingest_folder.py          step 1: the folder → per-page table (no text kept), page types
├── build_skeletons.py        step 2: page skeletons + vectors (text; image when present)
├── build_codebook.py         step 3: k-means note looks + look cards + naming sheet
```

```
├── build_catalogue.py        step 4: fingerprints, HDBSCAN, prototypes, limits, cards
├── apply_names.py            step 5: read the filled naming sheets → final model folder
├── evaluate.py               §6 checks, incl. "hide one type"
├── chart_types/              shared code, also copied into core-pipeline (see below)
│   ├── skeleton.py  fingerprint.py  distance.py  match.py  model_io.py
└── output/chart-types/v1/    the model (git-ignored)
```

```
core-pipeline/modules/chart_type/          copy of chart_types/ (a test fails if they differ,
                                           like the page canon)
core-pipeline/apps/worker/stages/chart_type.py   stage 16 adapter (reads DB, writes results)
core-pipeline/apps/worker/jobs/chart_types_weekly.py   the weekly refresh (§5.1)
core-pipeline/models/chart-types/vN/       the deployed model (Blob resources/ from phase 8)
```

The matcher and the trainer use **the same `skeleton.py`, `fingerprint.py` and `distance.py`**, so a chart gets the same fingerprint in training and in production. `ingest_folder.py` imports the page-leaf classifier from `core-pipeline/modules/` and does not copy it, so page types match the pipeline's exactly.

3.2 The page skeleton: a note's template without its patient

A progress note's **template** is the part that repeats across patients: section headers, field labels ("Chief Complaint:", "BP", "Assessment and Plan"), EMR boilerplate ("Electronically signed by", "Printed on", "Page 1 of 3", "last edited by"), and the order they come in. The patient part (names, dates, numbers, free text) changes every time. The skeleton keeps the first and drops the second:

1. **Vocabulary (built once, from the training folder):** a word is kept only when it is in the page/section canon, or it appears on pages of at least 5% of the charts **and** in charts from at least 3 different sources or deliveries. Names, places and rare words never pass that test. This is the same "generic words only" rule as the keyword mining. The vocabulary is part of the model and contains no patient data.
2. **Per page:** each Tesseract line is cut to its kept words; digits become `#` and dates become `<date>`; empty lines are dropped. Lines are tagged by their position on the page (top 15%, body, bottom 15%) and by shape (`label:` line, header-like line, list item, table row).
3. **Section-header sequence:** the canon matcher already used in stage 6 is run on the page's lines, giving an ordered list of canon headers.

From the skeleton, each Progress Note page gets a vector:

Block	Size	From
Template text	384	MiniLM (`models/semantic-model`, already on every host) on the skeleton's top + first 40 lines
Template words	256	TF-IDF over skeleton 1–3-grams, reduced by SVD fitted on train
Structure	24	header sequence (first 8 canon headers, one-hot by family), share of `label:` lines, of table rows, of lines in top / bottom bands, line count and mean length (binned)
Image (optional)	384	DINOv2-small on the deskewed, binarised 224 px page, only when §2's 90% rule holds

The blocks are L2-normalised separately and weighted 0.45 / 0.30 / 0.25 (with an image: 0.30 / 0.20 / 0.15 / 0.35).

Look cards for naming show a note look's most common skeleton lines, which contain only kept words. They show no patient text, so they can be reviewed on the Windows laptop. With images, a wireframe card is added.

3.3 Where each fingerprint part comes from

Part (§3 step 2)	Text-only folder	With `imaging/*.csv` / images	In production (stage 16)
Which pages count	page-leaf on Tesseract text: L1 = Progress Note; pass-1 blank/junk rules; word-overlap duplicates	`_final.csv`: `page_type`, `blank_junk_flag`, `is_duplicate`	`imaging_final`
Note looks	skeleton vectors (§3.2)	+ image block	same skeleton code on `ocr_results` (tesseract)
Note specialty	page-leaf / keyword specialty	`_final.csv` `specialty`	`imaging_final.specialty`
Note capture	**text proxies:** share of tokens not in a dictionary (garbled OCR ⇒ handwriting or poor scan), characters per page, fax header in the top band (`FAX`, `From:`, phone pattern, `Page x of y`), share of very short pages	`_hw_printed.csv` handwritten share and area, `_quality.csv` DPI, `_rotation.csv` tilt	`ocr_quality_results` + the same text proxies
Rest of the chart	page types of non-note pages → 6 document groups; page count	`_final.csv`, documents from `document_seq`	`imaging_final`
Source / retrieval method	`charts.csv` if given, else weight 0	same	`manifest_member_list`

Production uses the same proxies as training. The capture part is always computed from the text proxies. The image-based traits are added only if the model was trained with them (recorded in `version.json`).

3.4 Training steps (one command each)

Step	Does	Output	Person needed
1. `ingest_folder.py --root <folder>`	Reads each chart; splits the text into pages; page type, specialty, blank/junk, duplicates (from CSVs or computed); capture proxies. **Keeps no text**	`work/pages.parquet`	no
2. `build_skeletons.py`	Vocabulary, page skeletons, page vectors (images if the 90% rule holds)	`work/vectors.npz`, `vocab.json`	no
3. `build_codebook.py --k auto`	k-means on train note pages, k from 16–48 by silhouette (default 32), fixed seed; look limits; look cards	`codebook.npz`, `note_looks.xlsx`	**name each look**
4. `build_catalogue.py`	Fingerprints, distance matrix, HDBSCAN (`min_cluster_size=3`, precomputed), prototypes, limits, type cards	`catalogue.json`, `type_cards/`, `types.xlsx`	**name / merge types**
5. `apply_names.py`	Reads both sheets, applies names and merges, recomputes prototypes and limits	`output/chart-types/v1/`	no
6. `evaluate.py`	All checks in §6 on the val charts	`scorecard.html`	read it

`work/` lives next to the training data, not in the repo. It is the only place where per-page rows are kept, and those rows hold no text.

3.5 The model folder (one version)

```
output/chart-types/v1/
├── version.json            version, date, kind (initial | weekly | full), charts used, data hash,
│                           code version, seeds, weights, thresholds
├── codebook.npz            note-look centroids, per-look limits, TF-IDF + SVD for skeletons
├── vocab.json              the kept template words (generic only, no patient data)
├── note_looks.json         id, name, page count, limit, common skeleton lines
├── catalogue.json          per type: id (T-0001…), name, status, prototypes (fingerprints),
│                           limit, members, first/last seen; per-feature scalers
├── specialty_map.json      specialty → broad group
├── doc_group_map.json      page type → document group
└── type_cards/T-0001.html  mix, specialty, capture, sources, channels, 10 example charts
```

No page text, names, DOB or member ids are stored anywhere in it. Type ids are **never reused or renumbered**.

3.6 Stage 16 in the pipeline

- **Name:** `chart_type`, pass 1, after `imaging_final`. It is added to the `pipeline_stage` seed, so `only: ["chart_type"]` re-types a chart on its own in a few seconds, with no OCR.
- **Switch:** `CHART_TYPE_ENABLED=false` by default; `CHART_TYPE_MODEL_DIR=models/chart-types/current`.
- **Shadow first:** while `CHART_TYPE_SHADOW=true`, results are written but review-ui does not show them yet, and nothing else reads them.
- **Input:** Tesseract text from `ocr_results` (stage 2) plus `imaging_final` and `ocr_quality_results`. It needs no images and no Blob read, unless the model has an image block.
- **Cost per chart:** skeleton vectors for the Progress Note pages (about 1 s for 40 pages; about 6 s with an image block), then matching (under 50 ms).
- **Degraded runs are visible:**
  - with the model missing, the stage records `answer='skipped'` and its reason;
  - with fewer than 2 note pages, it records `no_notes`.
- **Monitoring:** the stage's time goes into `run_usage.stage_seconds` like any other stage.

3.7 Tables (Part 2 of `schema/schema.sql` until it ships, then Part 1)

These follow the CLAUDE.md naming rules. The concept note's names are kept where they fit.

Table	One row per	Columns
`chart_type_result`	chart × model version	`chart_id`, `type_code` (T-0001 / `new` / `no_notes`), `answer` (known / unsure / new / no_notes / skipped), `alt_type_code`, `distance` (d1), `margin` (d2 – d1), `notes_distance`, `confidence`, `flags` JSONB (`new_notes`, `edge`, `close_call`), `main_note_look`, `main_specialty`, `client_project_hash`, `retrieval_channel`, `model_version`, `is_current`
`page_note_look`	Progress Note page	`chart_id`, `page_id`, `note_look_code`, `distance`, `is_new_look`, `model_version`
`chart_type_held`	held chart	`chart_id`, `status` (held / in_candidate / rare / released), `candidate_code`, `held_since` (= `created_at`), `delivery_key` (`run_id`/`batch_id`)
`chart_type_version`	model version	`version`, `kind`, `charts_used`, `data_hash`, `types_added`, `types_merged`, `looks_added`, `released_by` (`user_id`)

The catalogue and codebook stay as files in the model folder. The DB holds only answers and history, so rolling back a model is a file change.

4. Scoring: how each number is computed

4.1 Page → note look

For a Progress Note page with combined vector `p` = the §3.2 blocks (template text, template words, structure, plus image when present), each L2-normalised and weighted, then L2-normalised again:

- `d_look(p, c) = 1 − cos(p, c)` to each codebook centroid `c`.
- The page takes the nearest look. It is a **new note look** when that distance is above the look's limit (95th percentile of its own training pages' distances, × 1.25).

4.2 The seven part distances (each in [0, 1])

Part	Distance between charts A and B	Weight
Note-look mix	Jensen–Shannon distance (base 2) between the two 32-bin shares. A chart with two big looks keeps both	0.40
Note overall look	0.7 · min(1, (1 − cos(μA, μB)) / τ) + 0.3 · |sA − sB| / s_max, where μ is the mean page vector, s the mean spread, and τ the 95th percentile of all train pairs	0.15
Note specialty	Jensen–Shannon distance between the 6-group shares	0.15
Note capture	Gower: mean over the capture traits (§3.3: text proxies, plus image traits when trained with them) of |a − b| / (train range), clipped to 1	0.15
Rest of the chart	0.8 · JS distance of the document-group shares + 0.2 · |log pages ratio| / log 10 (clipped)	0.05
Source	0 if the same client+project hash, else 1	0.05
Retrieval method	0 if the same channel, else 1	0.05

- **Chart distance** $D(A,B) = \Sigma\ w_i \cdot d_i$, in [0, 1].
- **Notes-only distance** `D_notes(A,B)` = the first four parts re-weighted to sum to 1 (0.40/0.85, 0.15/0.85 …). It alone decides **new** (§3: "never alone").

4.3 Chart → type

For each type T with prototypes $P_1 \ldots P_n$ ($n \le 10$):

- $d(chart, T) = \min$ over prototypes of $D(chart, P_j)$. The notes-only `dn(chart, T)` is computed the same way.
- **Prototypes:** the members with the lowest mean distance to the rest of their type (medoids), up to 10, chosen greedily so they cover the type and are not all alike.
- **Limit** `L_T` = 95th percentile of the members' leave-self-out distance to their nearest prototype, clipped to [0.15, 0.45].

Take T1 (nearest, `d1`) and T2 (runner-up, `d2`). The answer follows §5 of the concept note:

Condition	Answer	Flag
fewer than 2 kept Progress Note pages	`no_notes` (typed by the rest of the chart, small separate catalogue)	—
d1 ≤ L_T1 and d2 − d1 ≥ 0.05	**known** T1	—
d1 ≤ L_T1 and d2 − d1 < 0.05	**unsure** T1, alt T2	`close_call`
L_T1 < d1 ≤ 1.25 · L_T1	**unsure** T1	`edge`
dn > 1.25 · L_T for **every** type	**new**, held	—
otherwise (only source or channel pushes it out)	**unsure** T1	`edge`
any answer, with 30%+ of note pages being new note looks	unchanged	`new_notes`

4.4 One confidence number (the repo's 0–1 scale: ≥ 0.8 good, < 0.5 review)

```
fit    = 1                                  if d1 ≤ 0.5·L
       = 1 − 0.5 · (d1 − 0.5·L) / (0.5·L)   if 0.5·L < d1 ≤ L        (1 → 0.5)
       = 0.5 · (1.25·L − d1) / (0.25·L)     if L < d1 ≤ 1.25·L       (0.5 → 0)
margin = min(1, 0.6 + 0.4 · (d2 − d1) / 0.10)
confidence = fit × margin                   (new / no_notes: blank)
```

A chart in the core of its type with a clear runner-up scores 1.0. A chart on the limit scores 0.5. A close call loses up to 40%. These constants start here and are tuned on the val charts with the other thresholds (§6.3). The **flags**, not the number, decide the review.

4.5 Starting values (all in `version.json`, tuned on val)

Constant	Start
Weights	0.40 / 0.15 / 0.15 / 0.15 / 0.05 / 0.05 / 0.05
Limit clip	0.15 – 0.45
Close-call margin	0.05
Edge band	1.25 × limit
New note looks for `new_notes`	30% of the chart's note pages
Codebook k	32 (16–48 by silhouette)
HDBSCAN	`min_cluster_size=3`, `min_samples=2`, `cluster_selection_method='leaf'`

---

5. Refresh

There are two refreshes (weekly and full) plus a weekly drift report, and **neither refresh changes an existing type id or rewrites an earlier answer**. Every answer keeps the `model_version` that gave it.

5.1 Weekly (on the VM, no images, no person until the review)

Job: `python -m apps.worker.jobs.chart_types_weekly`. It runs on the first trigger after Monday 06:00, the same way the log purge runs once a day.

1. **Collect** the held charts (`chart_type_held.status = 'held'`) and the new note looks from `new_notes` charts. Their fingerprints and embeddings are already stored, so nothing is re-read from Blob.

2. **Group** them: HDBSCAN on `D`, with at least 3 charts per group. New note looks are grouped one level down with k-means on the new pages, and each group needs at least 20 pages from 2 or more charts.
3. **Candidate rules** (concept note §6):
    - 3+ charts from 2+ deliveries (`delivery_key`);
    - spread ≤ the median type limit;
    - nearest known type at `dn > 1.25 · L`, otherwise it is a **variant** of that type;
    - a fax-only group is compared with the known types on the notes-only distance, and if it matches one, it is a variant.
4. **Review sheet** `chart_type_review_<date>.xlsx`, with links to the type cards. Each row is decided as Confirm + name, Merge into T-xxxx, or Reject. A reviewer can fill it in on the Windows laptop. Later it becomes a review-ui admin page.
5. **Release** `vN+1` (`python -m apps.worker.jobs.chart_types_weekly --apply <sheet>`):
    - confirmed types get the next ids, and merges add prototypes;
    - types that gained 20+ charts get their prototypes and limits recomputed;
    - every held chart is re-matched (`only: ["chart_type"]`, which takes seconds);
    - the new answers get `is_current = true` and the old ones are kept.
6. **Leftovers:**
    - a chart held 90 days with no group becomes `rare`;
    - rejected groups are remembered (their centroid is stored), so they are not proposed again unless 3 new charts join them.

5.2 Full rebuild with a new or larger folder, about quarterly

```
python build_all.py --root "<folder>" --base output/chart-types/v7
```

This is a **full rebuild**: a new vocabulary, codebook and HDBSCAN run, on the training folder (old charts plus any you add) and the charts confirmed since the last rebuild. After that:

- **Look ids are kept** by matching new centroids to old ones (Hungarian matching on cosine; a match needs distance < 0.15).
- **Type ids are kept** by matching new clusters to old types on shared member charts (Jaccard ≥ 0.5). Unmatched clusters get new ids. Old types with no match become `dormant`, never deleted.
- **Sign-off:** it is released only after a person reads the diff report: types split, merged, added and gone, and charts whose type changed.

Triggers:

- once a quarter;
- the codebook has grown by 25% since the last full rebuild;
- the weekly drift report alerts two weeks running;
- a new page-type or OCR model is released, because fingerprints depend on page types and the OCR layout.

5.3 Drift report (weekly, with the job)

- **Shares:** the share of charts answered new, unsure and `new_notes`, overall, per client and project hash, and per channel, against the previous 4 weeks.
- **Alerts:**
    - new above 10% of the week's charts, or above 25% for one client;
    - unsure above 20%;
    - the median confidence of a type drops by 0.1.
- **Per type:** charts this week, and pipeline accuracy where ground truth exists. Once types exist, accuracy can be broken down by type.

---

6. How the model is scored (quality, not one chart)

6.1 Every build writes `scorecard.html`

Check	How	Target
Codebook tightness	Silhouette of the k-means looks on train pages	≥ 0.15
Cluster validity	DBCV of the HDBSCAN result; noise share	DBCV ≥ 0.3; noise ≤ 30% of train charts
Types are real	Person: 10 charts' note skeletons per type	≥ 90% one kind
Note looks are real	Person: 20 page skeletons per look	≥ 90% one template
Text–production match	20 charts present in the folder **and** run through the pipeline: fingerprint from the folder vs from the DB	same type for ≥ 95%
Known stays known	Val charts whose train neighbours are in a type: share answered new	≤ 5%
Wrong type	Val charts with a `label`: share filed under a type of another label	≤ 5%
Agreement with your labels	ARI and NMI between types and `label` on labelled charts	ARI ≥ 0.6
New gets noticed	**Hide one type** (every type with 5+ charts): rebuild without it, run its charts	≥ 80% new, and they form **one** candidate
Capture doesn't split	Labelled families arriving by 2+ channels: share in one type	≥ 90%
Stable	10 rebuilds on 80% samples; ARI between runs	mean ≥ 0.8
Fast	Matching one chart on CPU, after embeddings	< 1 s

6.2 What your `label` column buys

With no labels, "types are real" rests on the person looking at cards. With labels on even 30–50 charts, ARI/NMI and "wrong type" become measured numbers. The tuning in §6.3 can then optimise against them.

6.3 Tuning (on val only, never on train)

A small grid search runs over the weights (the notes total stays fixed at 0.85), the limit clip, the margin and the edge band. It looks for the best "known stays known" and "wrong type", subject to "new gets noticed" ≥ 80%. The chosen values go into `version.json`.

---

7. Build plan (maps to C0–C6 of the concept note)

Status (2026-11-07): C0 part 1 is built in `training/chart-clustering/` (`ingest_folder.py --root <folder>`: text split, pipeline CSV labels when present, capture traits from the text; writes `work/pages.csv` and `work/charts.csv`, no text kept). Next: page-leaf on text for charts without `_final.csv`, then C0b.

Step	Work	Size	Done when
C0	`ingest_folder.py` (text split, page-leaf on text, blank/junk + duplicates, capture proxies; CSVs when present)	2 days	One row per page for the 500 charts; page types agree with the pipeline's on charts that have `_final.csv` (≥ 95%)
C0b	Vocabulary + skeletons + page vectors (`build_skeletons.py`)	2 days	Skeletons of 50 random pages contain no name, date or ID (checked by a person)
C1	Codebook + look cards + naming sheet	1–2 days	Every look named; codebook v1
C2	Fingerprint, distances, HDBSCAN, prototypes, limits, type cards	2–3 days	Every type named; catalogue v1
C3	Matcher + confidence + flags; `evaluate.py`; tuning	2 days	"Known stays known" and "wrong type" met
C4	Stage 16 in shadow; tables (Part 2); `CHART_TYPE_*` in both `.env` files	2 days	7 test charts run with **no change** to existing outputs
C5	Weekly job, review sheet, release, drift report; full rebuild with id mapping	3 days	A candidate can be confirmed, merged or rejected; ids survive a full rebuild
C6	"Hide one type" test; review-ui admin page (types, held, review); switch off shadow	2–3 days	"New gets noticed" met; you approve

That is about 3½ weeks of work. C0–C3 run entirely in `training/` on the Mac and touch nothing on the VM, so they can start as soon as the folder is on the SSD.

---

8. Commands

Mac (training)

```
cd training/chart-clustering
python3.12 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt
ROOT="../../core-pipeline/outputs/charts"   # or any folder in the same format (+ optional charts.csv)
python build_all.py --root "$ROOT"            # steps 1–4, then fill the two sheets
python apply_names.py && python evaluate.py
scp -r output/chart-types/v1 user@VM:~/advantmed-imaging-pipeline/core-pipeline/models/chart-types/
```

Windows laptop (Command Prompt): the reviewer fills in the sheets and copies the model to the VM

```
start note_looks.xlsx
start types.xlsx
:: name the looks and types, save, then:
scp -r output\chart-types\v1 user@VM:~/advantmed-imaging-pipeline/core-pipeline/models/chart-types/
```

Ubuntu VM (bash): switch on in shadow, re-type a chart, run the weekly job by hand

```
cd ~/advantmed-imaging-pipeline/core-pipeline
ln -sfn v1 models/chart-types/current
# .env: CHART_TYPE_ENABLED=true  CHART_TYPE_SHADOW=true
docker compose up -d
curl -X POST localhost:8001/api/charts/run -H 'Content-Type: application/json' \
```

```
          -d '{"input_type":"blob","container_name":"imaging-
pipeline","input_path":"Raw_Input/Run1/Batch1/DEID_PNGs","chart_name":"52743839_44976074","only":
["chart_type"]}'
        docker compose exec api python -m apps.worker.jobs.chart_types_weekly            # propose
        docker compose exec api python -m apps.worker.jobs.chart_types_weekly --apply
chart_type_review_2026-11-16.xlsx
```

From phase 8 the model folder is published to Blob `resources/models/chart-types/` with the manifest, and the `scp` step goes away.

---

9. Decisions needed from you

1. **The folder:** about 500 charts with Tesseract text. Will most of them also have `imaging/*.csv` (run through the pipeline) or page images? Family labels for even 30–50 of them help a lot.
2. **Source and channel:** can a `charts.csv` give `client_id` / `project_id` / `rr_method` per chart? Without it, those parts are left out of training (weight 0).
3. **Delivery:** is `run_id` + `batch_id` the right "delivery" for the 2-deliveries rule, or should one large delivery from a new client be enough?
4. **Reviewer:** who reviews, and on which day? Should the review start as the sheet and become a review-ui page in C6?
5. **Full rebuild:** quarterly plus the triggers in §5.2, or something else?
6. **Text-first fingerprint** (§3.2), with images optional: OK? It keeps training and production identical and needs no extra OCR or images.